



PILLAI COLLEGE OF ENGINEERING

PROJECT REPORT

Student Productivity & Study Planner System

**Submitted in partial fulfilment of the requirements for the degree of
Bachelor of Technology**

In

ELECTRONICS AND TELECOMMUNICATION ENGINEERING

Submitted by: Darshan Bajirao (ETCA511)

Project Guide	Dr. Aboli Khedkar
Subject	Java Programming

**Department of Electronics and Telecommunication Engineering
Academic Year 2026–2027**

Mentors Sign:

INDEX

Sr No.	Content	Page No.
1.	Abstract and Introduction	3
2.	Objectives and Scope	4
3.	System Design and Methodology	5–6
4.	Object-Oriented Design and Technical Concepts	7
5.	Java Implementation Details	8–10
6.	Functional Modules	11
7.	Data Storage and Security	12
8.	Productivity Calculation and Statistics	13
9.	Results and Discussion	14
10.	Conclusion and Future Scope	15
11.	References	16

Abstract

The Student Productivity & Study Planner System is a Java-based desktop application developed using Java Swing and AWT. It provides students with a single workspace for organizing subjects, academic tasks, study schedules, notes, focused study sessions, reminders and productivity statistics. The application is designed as a self-contained local system and does not require a web server, cloud database or internet connection.

The project demonstrates practical use of object-oriented programming, collections, enumerations, file handling, Java serialization, exception handling, event-driven GUI programming, custom GUI painting and timer-based functionality. Local user accounts and workspace information are persisted using Java serialization, while a SHA-256 hash is used for local password storage.

1. Introduction

Students often manage assignments, examinations, subjects, notes and study sessions using separate tools. This can make it difficult to maintain a single view of academic priorities and progress. The proposed system combines these activities in one Java desktop workspace.

The application provides modules for authentication, dashboard monitoring, subject management, task management, weekly planning, a 25-minute Pomodoro focus timer, notes, statistics and reminders. The dashboard summarizes completed tasks, pending tasks, study hours and calculated productivity.

1.1 Problem Statement

To develop a Java desktop application that helps students organize academic information, plan study time, track tasks and review productivity from a single local workspace.

1.2 Scope

- Centralized academic workspace for subjects, tasks, notes and study sessions.
- Weekly study planning and focused 25-minute sessions.
- Task priorities, statuses and due dates.
- Productivity statistics and subject-wise study-time distribution.
- Deadline reminders and local persistence.
- Standalone operation without a web server or internet dependency.

1.3 Importance of the Project

The project provides a practical example of how Java programming concepts can be integrated into a useful desktop application. It connects user-interface events with object-oriented data models, persistence and simple analytical calculations, making it suitable for demonstrating Java programming concepts in an academic setting.

2. Objectives and Scope

2.1 Objectives

- Provide a centralized study-planning workspace for students.
- Allow users to create and maintain subject records.
- Track academic tasks with priorities, statuses and deadlines.
- Maintain a weekly study timetable and focused study sessions.
- Provide a 25-minute Pomodoro-style focus timer.
- Store subject-related academic notes.
- Display task completion, study hours and productivity statistics.
- Provide reminders for upcoming and overdue work.
- Demonstrate important Java concepts through a practical application.

2.2 Project Scope

The project is intentionally implemented as a desktop Java application. It does not require a web server, cloud database or internet connection. User and workspace information is stored locally in a serialized data file. The modular design leaves room for later cloud, mobile and calendar integrations.

2.3 Hardware and Software Requirements

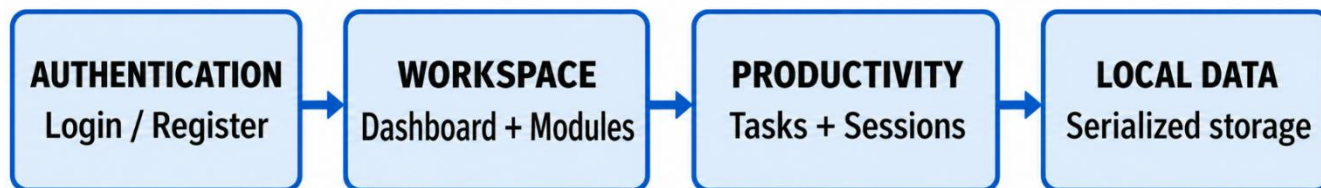
Category	Requirement
Hardware	Any computer capable of running a modern Java desktop application.
Operating System	Windows, Linux or macOS.
Java	JDK 17 or newer; JDK 21 is recommended.
IDE	Visual Studio Code, IntelliJ IDEA or Eclipse.
Libraries	Java standard libraries; Swing/AWT; no external libraries required.
Storage	Local writable project directory for the application data file.

3. System Design and Methodology

The system follows a modular desktop architecture. The model layer stores application entities, the service layer provides persistence and productivity calculations, and the user-interface layer provides Swing panels and dialogs.

3.1 High-Level Architecture

Student Productivity & Study Planner — System Architecture



The application starts from the login/registration interface. After successful authentication, the user enters a common workspace containing the Dashboard and feature modules. Task status, study sessions, subjects, notes and plans are maintained in the user's workspace and persisted locally.

3.2 Main System Components

- Authentication: creates accounts and validates local login credentials.
- Main Workspace: provides common navigation and the application window structure.
- Model Layer: represents User, Subject, Task, StudySession, StudyPlan, Note, Workspace and AppData.
- Service Layer: handles data persistence and productivity calculations.
- User Interface Layer: implements the login screen, dashboard and all functional panels using Swing/AWT.

3.3 Main Processing Flow

Application Processing Flow



4. Java Implementation Details

The project uses Java Swing and AWT for the graphical interface. The application is organized into model, service and UI components, with Main.java acting as the entry point.

5.1 Package and File Structure

Package / File Group	Purpose
src/Main.java	Starts the application and installs the application theme.
src/model	Application entities and serialized data structures.
src/service	Persistence and productivity calculation logic.
src/ui	Login screen, main workspace and all feature panels
RUN_WINDOWS.bat	Convenience launcher for Windows.
data/student_planner.dat	Local serialized application data created at runtime.

5.2 User Interface

The application uses a dark theme with a consistent technology-oriented visual style. The interface contains a login screen and a main workspace with persistent navigation for Dashboard, Subjects, Tasks, Study Planner, Notes, Statistics and Reminders.

- The login screen supports registration and sign-in.
- The main workspace uses a persistent sidebar and common window structure.
- Cards, buttons and tables use consistent typography and spacing.
- Workspace navigation uses larger, readable labels with static active-state styling.
- The interface is designed for a desktop display and does not depend on browser rendering.

5.3 Entry Point

Main.java → AppTheme.install() → LoginFrame → authentication → MainFrame

5.4 Local Data Persistence

The DataStore service saves AppData to data/student_planner.dat. Java ObjectOutputStream is used to serialize the application state and ObjectInputStream is used to restore it. A temporary file is written before replacing the main data file, reducing the risk of leaving a partially written save file.

If saved data cannot be loaded, the application reports the issue and starts with a fresh AppData object rather than stopping at startup. This provides a safer local demonstration workflow.

5.5 Password Handling

Local account passwords are not stored as plain text. DataStore uses the SHA-256 MessageDigest algorithm to create a hexadecimal password hash before storing the User object. During login, the entered password is hashed again and compared with the stored hash.

This is suitable for demonstrating basic local password handling in a college project; it is not intended to replace production-grade password storage systems such as salted, adaptive password hashing.

5.6 Productivity Service

$\text{Productivity (\%)} = \text{Completed Tasks} / \text{Total Tasks} \times 100$

$\text{Study Hours} = \text{Total Logged Study Minutes} / 60$

The ProductivityService calculates completed tasks, pending tasks, total study minutes, productivity percentage, tasks due within three days and subject-wise study minutes.

5.7 Data Update Behaviour

- Adding, editing, deleting or completing a task updates the workspace records.
- Logging a study session adds a StudySession object and updates study-hour statistics.
- Subject and note changes remain associated with the active user's workspace.
- The application saves modified workspace data locally so it can be restored on the next launch.

5.8 Study Planner and Pomodoro Timer

The Study Planner combines timetable information with a focused study session timer. The project uses a 25-minute focus period as the standard session length.

Function	Implementation
Timer display	Swing label showing the remaining time.
Session duration	25 minutes.
Timer scheduling	ScheduledExecutorService in the planner module.
Session logging	A StudySession object is added to the workspace after the focus session.
Persistence	The workspace is saved after the session is logged.
Navigation	Users can move between planning and other workspace modules.

6. Functional Modules

The application is divided into separate functional modules so that each academic activity can be managed from the same workspace.

Module	Main Function
Authentication	User registration, local login validation and workspace association.
Dashboard	Summary of completed tasks, pending tasks, study hours and productivity.
Subjects	Create, edit and delete subject records including code, teacher and target study hours.
Tasks	Create academic tasks with subject, priority, status and due date; update task progress.
Study Planner	Plan study activity and use the 25-minute focus timer to log sessions.
Notes	Maintain subject-related academic notes within the workspace.
Statistics	Display task completion, study hours, productivity and subject-wise study-time information.
Reminders	Identify incomplete overdue work and upcoming deadlines.

6.1 Module Interaction

The modules are connected through the active user's Workspace. For example, a task can be associated with a subject, a study session can be associated with a subject, and the Statistics module can use these stored records to produce subject-wise study-time information.

6.2 User Workflow

1. Register or sign in.
2. Open the Dashboard to review current progress.
3. Add subjects and define target study hours.
4. Create tasks with priorities and deadlines.
5. Use the Study Planner for focused sessions.
6. Record notes and review reminders.
7. Open Statistics to evaluate task completion and study activity.

7. Data Storage and Security

7.1 Local Storage Model

Application data is represented by AppData. It contains maps for users and workspaces. Each Workspace contains lists for subjects, tasks, study sessions, notes and study plans. Because these model classes implement Serializable, the object graph can be stored as one local data file.

7.2 Data Flow

User Action → UI Panel → Workspace / AppData → DataStore.save() → student_planner.dat

When the application starts, DataStore.load() attempts to restore the previous state. During operations that modify the workspace, the application calls the save functionality so that changes persist locally.

7.3 Basic Security Measures

- Passwords are transformed using SHA-256 before being stored.
- The application is designed for local/offline use and does not transmit account information over the internet.
- The data file remains in the application's local data directory.
- Application data is separated by user workspace within the stored object graph.

7.4 Security and Storage Limitations

- Java serialization is appropriate for a local academic project but is not a substitute for a production database.
- SHA-256 alone is not a modern password-storage scheme for production authentication.
- There is no cloud synchronization or multi-device account service.

Local data protection depends partly on operating-system file permissions and access to the project directory.

8. Productivity Calculation and Statistics

The application provides a simple quantitative view of study activity. The calculations are intentionally transparent so that students can understand how the dashboard values are produced.

8.1 Task Completion

Completed Tasks = count of tasks whose status is COMPLETED

Pending Tasks = count of tasks whose status is not COMPLETED

8.2 Productivity Percentage

Productivity = Completed Tasks $\times$ 100 / Total Tasks

If no tasks exist, productivity is displayed as 0% to avoid division by zero.

8.3 Study Hours

Study Hours = $\sum$ StudySession.minutes / 60

Study sessions are also grouped by subject. The Statistics panel uses these grouped values to display subject-wise study time.

8.4 Deadline Analysis

The service identifies incomplete tasks whose due dates fall from the current date through the next three days. The Reminders panel also identifies incomplete overdue tasks and displays upcoming academic work within its configured window.

8.5 Interpretation

These calculations provide a lightweight academic productivity view rather than a clinical or scientific measure of student performance. The purpose is to give the user an understandable summary of task completion and logged study activity.

9. Results and Discussion

9.1 Functional Result

The developed application provides a complete local student-planning workflow: a user can register, sign in, enter a workspace, maintain subjects, manage academic tasks, plan study sessions, write notes, review statistics and check reminders.

9.2 Interface Result

The final interface uses a dark technology-oriented visual style. The design uses a persistent sidebar, consistent spacing, rounded cards, accent colors and readable typography. The application also includes a custom login screen and a consistent window-control bar.

9.3 Representative Functional Scenarios

Scenario	Expected Application Behaviour
New user registration	Creates a local User and workspace and saves the data.
Valid login	Opens the user's workspace.
Add subject	Adds a Subject record to the workspace.
Add task	Adds a Task with subject, priority, status and due date.
Complete task	Changes task status to COMPLETED and updates productivity.
Log study session	Adds study minutes and updates study-hour statistics.
Add note	Stores academic note content in the workspace.
Open reminders	Shows relevant incomplete upcoming/overdue tasks.

9.4 Discussion

The project demonstrates how several Java concepts can be integrated into one useful desktop application. Its main strength as a mini project is the connection between GUI events, object-oriented data models, persistence and simple analytical calculations.

10. Conclusion and Future Scope

10.1 Conclusion

The Student Productivity & Study Planner System demonstrates a practical Java Swing desktop application for academic planning. It combines authentication, subject management, task tracking, study planning, notes, reminders and productivity statistics within one local workspace.

The project also demonstrates important Java programming concepts including classes and objects, encapsulation, enumerations, collections, serialization, file handling, exception handling, event listeners, custom GUI painting and timer-based functionality. The modular structure makes the project suitable for a Java mini-project demonstration and viva.

10.2 Limitations

- The application is primarily a local desktop system.
- Data is stored in a local serialized file rather than a database or cloud service.
- There is no multi-device synchronization.
- Productivity is represented using simple task-completion and study-time calculations.

10.3 Future Scope

- Cloud synchronization and secure online backup.
- Mobile or web version of the planner.
- Google/Outlook calendar integration.
- Desktop or push notifications for reminders.
- Advanced analytics, weekly trends and goal tracking.
- AI-assisted study-plan generation and task prioritization

11. References

The following references support the Java technologies and development concepts used in the project.

1. Herbert Schildt, *Java: A Beginner's Guide*, McGraw Hill Education.
2. E. Balagurusamy, *Programming with Java*, McGraw Hill Education.
3. K. S. Saini, *Java Programming*, Kalyani Publishers.
4. R. Nageswara Rao, *Core Java: An Integrated Approach*, Dreamtech Press.
5. Java Programming notes and study material provided during classroom lectures.
6. Java Swing and GUI programming concepts studied as part of the Java Programming syllabus.

Project Summary

Project	Student Productivity & Study Planner System
Language	Java
GUI	Java Swing / AWT
Persistence	Java Serialization
Authentication	Local account + SHA-256 password hashing
Core Modules	Dashboard, Subjects, Tasks, Study Planner, Notes, Statistics, Reminders
Deployment	Standalone desktop application
External Dependencies	None